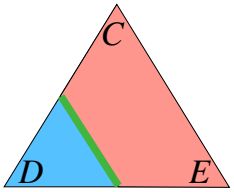
		Player Y		
		C	D	E
Player X	A	1	0	1
	B	0	1	0

(a) A common-payoff game between two players.



(b) The strategic equivalence relation over player Y's policy space. Vertices represent pure strategies.

Figure 2: Figure 2a shows a common-payoff game between two players with multiple Nash equilibria. In Figure 2b, the red region (upper right) represents the policies for which A is a best response, the blue region (lower left) represents the policies for which B is a best response, and the green region (separating line, which technically has zero width) represents the policies for which both A and B are best responses.

($\Longleftarrow$) Let $B = \text{BR}_i^{-1}(A)$ be the preimage of A, an arbitrary set in the domain of BR_i. By definition, for any co-policy $\pi_{-i} \in \Pi_{-i}$, we have $\text{BR}_i(\pi_{-i}) = A$ if and only if $\pi_{-i} \in B$. Therefore, any strict superset of B must contain a policy that has a best-response set different from A, which would introduce strategic ambiguity. Therefore, B contains no irrelevant information.

($\Longrightarrow$) Now let B be a strategically unambiguous subset of Π_{-i} that contains no irrelevant information. Since B is strategically unambiguous, by definition, all policies in B must have the same best-response set; call this set A. Since B contains no irrelevant information, it must contain *all* policies in Π_{-i} that induce best response A. Therefore, $B = \text{BR}_i^{-1}(A)$ and is in the preimage of BR_i. □

In this sense, the preimage of BR_i gives us the coarsest (fewest element) partitioning of Π_{-i} into subsets that are strategically unambiguous. We call the equivalence relation induced by this partitioning the *strategic equivalence relation*.

Definition 4.3. The *strategic equivalence relation* (SER) $\sim_i$ for player i is the equivalence relation over the co-policy space Π_{-i} such that $\pi_{-i} \sim_i \pi'_{-i}$ if and only if $\text{BR}_i(\pi_{-i}) = \text{BR}_i(\pi'_{-i})$. We refer to the equivalence classes of the SER as *strategic equivalence classes* (SECs), which partition the co-policy space. We write $[[\pi_{-i}]]$ to denote the SEC that contains π_{-i}.

Figure 1 shows how the best-response function partitions the co-policy space into SECs. The SECs are exactly the strategically unambiguous subsets of Π_{-i} that contain no irrelevant information. If two policies fall in the SEC, then they must induce the same best response from agent i, by definition. Knowledge that allows agent i to differentiate between policies within the same class is extraneous, since it has no affect on agent i's choice of optimal policies. In this sense, which equivalence class contains π_{-i} is the minimum knowledge that agent i needs to compute their set of best responses.

Consider the payoffs in Table 2a. Suppose player X knows that player Y's policy takes the form $w_1 C + w_2 E$. Notice that although different weights w_1, w_2 give distinct policies, they all fall into the same SEC because they induce the same best response, A, from player X. All policies that are more likely to choose D than C or E (combined) fall into a separate SEC and policies that choose D and C or E (combined) with equal probability fall in a third. Figure 2b shows player Y's mixed policy space and its three SECs. In order for player X to play optimally, they only need to know which of the three SECs player Y's policy falls in.

SERs partition policies into classes depending on the best response that they induce, regardless of the payoff that those policies induce. It is possible for two policies to be strategically equivalent $\pi_{-i} \sim_i \hat{\pi}_{-i}$ (i.e., they induce the same best response from agent i), while inducing different payoffs. A *valued strategic equivalence relation* is a refinement of an SER that differentiates such policies.

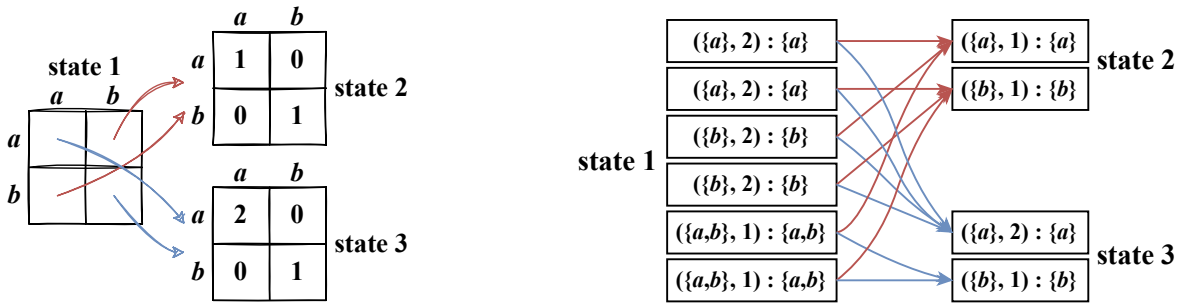
Definition 4.4. A *valued strategic equivalence relation* (VSER) $\overset{\text{v}}{\sim}_i$ for player i is an equivalence relation over the co-policy space Π_{-i} such that $\pi_{-i} \overset{\text{v}}{\sim}_i \hat{\pi}_{-i}$ if and only if,

(i) $\text{BR}_i(\pi_{-i}) = \text{BR}_i(\hat{\pi}_{-i})$, and

(ii) $R(\pi_i, \pi_{-i}) = R(\pi_i, \hat{\pi}_{-i})$ for all $\pi_i \in \text{BR}_i(\pi_{-i})$.

5. Computing strategic equivalence relations

In the most general setting, our theory applies to strategic equivalence relation (SER) over the space of *mixed* policies. Even though the number of SECs is still guaranteed to be finite in the case of mixed policies (see Lemma 2 in Appendix A for more details), we have a stronger bound in the case of pure policies, where the number of strategic equivalence classes is upper-bounded by the number of pure policies (see Appendix B.1 for more details). Moreover, it is computationally easier to only compute the set of best-response policies over pure strategies in DecPOMDPs (Szer & Charpillet, 2006). For these reasons, we limit ourselves to computing SERs over pure strategies. For similar com-



(a) A two-step dynamic game. The agents first take a joint action that transitions them into one of two final states, and then they receive a joint payoff according to their second joint action.

(b) The VSER DAG $\mathcal{G}_i$ for both players. Each box represents a node in $\mathcal{G}_i$ with the format $[(A_i^*, V^*) : \hat{\Pi}_{-i}]$ where A_i^* is the best-response set, V^* is the best-response value, and $\hat{\Pi}_{-i}$ are the one-step co-policies, the actions the co-policies in the equivalence class prescribe at state s.

Figure 3: A dynamic game along with the VSER DAG $\mathcal{G}_i$ for both players (since the game is symmetric).

putational reasons, we restrict ourselves to the domain of common-payoff games, although all of our theory extends to general-sum games as well.

In Section 5.3 we compute the VSER over the set of best-response policies in DecPOMDPs. In Appendix B.6, we give a simplified (and faster) algorithm for computing VSERs over Nash equilibrium in fully-observed DecPOMDPs. For more details and rationale behind computing strategic equivalence relations over subsets of the full policy space, see Appendix B.4.

5.1. Normal-form games

As a warmup, we quickly explain how to compute the strategic equivalence relation (Definition 4.3) of a normal-form game (i.e., a DecPOMDP with a single state). In order to compute the strategic equivalence relation for agent i, we just need to enumerate the pure policies $\pi_{-i} \in \Pi_{-i}$, compute the best responses $\text{BR}_i(\pi_{-i}) = \arg\max_{\pi_i \in \Pi_i} R(\pi_i, \pi_{-i})$ for each one, and group the policies that induce the same best responses.

5.2. Strategic equivalence graphs

In DecPOMDPs with multiple states, instead of explicitly computing the policy within every valued strategic equivalence class (VSEC) (see Definition 4.4), we compute a more compact structure that takes advantage of the inherently recursive nature of optimal policies. This will entail computing a directed acyclic graph $\mathcal{G}_i$ as follows for each agent i.

For simplicity, let's first restrict ourselves to the case of a *fully-observed* DecPOMDP. For each subgame starting in state s there exist multiple nodes in $\mathcal{G}_i$. Each node conceptually stores a single class of the *intermediate VSER*, the VSER over policies restricted to the subgame starting at state s. Each node explicitly contains the,

1. The state s that it's associated with;
2. The *one-step co-policies*: the actions that the co-policies in the SEC prescribe at state s;
3. The set of best-responses A_i^* for agent i in state s;
4. The value V^* that they induce from agent i in state s;
5. The edges to children nodes associated with the successor states of s that continue the VSEC in the future.

Item (5) is required because the value of agent i's actions at state s are dependent on what the policies of all agents prescribe to future states. Each of these children nodes contains the information on how the VSEC continues in the future.

Each VSEC over the complete policy space is given by a root node of $\mathcal{G}_i$ (which are each associated with the starting state of the DecPOMDP) along with all of its children. For simplicity and to give computational speedup, nodes that share the same history, best-response actions, and best-response value can be merged, turning $\mathcal{G}_i$ into a multigraph (i.e. a graph that is allowed to have multiple edges between the same two nodes).

Figure 3b shows $\mathcal{G}_i$ for the simple two-step game depicted in Figure 3a. Each box represents a node in $\mathcal{G}_i$ along with it's information in the format $[(A_i^*, V^*) : \hat{\Pi}_{-i}]$, where $\hat{\Pi}_{-i}$ are the one-step co-policies. See Appendix B.2 for a detailed explanation of $\mathcal{G}_i$ in this example.

The primary change to $\mathcal{G}_i$ in partially-observed DecPOMDPs is that nodes must be associated with histories, not states, since optimal policies are computed over history-dependent beliefs. Let $\Pi_{-i}^{h_i} \subset \Pi_{-i}$ denote the subset of co-policies that are *consistent* with history h_i, i.e., there exists some random outcomes that make h_i a valid

history. The nodes associated with history h_i in $\mathcal{G}_i$ store intermediate VSERs $\overset{\text{v}}{\sim}_{h_i^t}$ over the subsets $\Pi_t^{h_i}$ based on the best response they induce from agent i, given that they've already observed h_i. Formally, for $\pi_{-i}, \pi'_{-i} \in \Pi_{-i}^{h_i}$, we have $\pi_{-i} \overset{\text{v}}{\sim}_{h_i^t} \pi'_{-i}$ if,

$$\begin{aligned} &\text{BR}_{h_i}(\pi_{-i}) = \text{BR}_{h_i}(\pi'_{-i}), \text{and} \\ &R(\pi_i, \pi_{-i}) = R(\pi_i, \pi'_{-i}), \forall \pi_i \in \text{BR}_{h_i}(\pi_{-i}), \end{aligned} \tag{2}$$

where $\text{BR}_{h_i} : \Pi_{-i}^{h_i} \to \Pi_i^t$ maps to the set of best-responses for agent i from timepoint t onward, given history h_i.

Each node in $\mathcal{G}_i$ conceptually stores a single class of the intermediate VSER $\overset{\text{v}}{\sim}_{h_i^t}$. However, each node does not specify the full policies in that class. Each node contains the one-step co-policies, the partial policies for agents $-i$ that prescribe actions to the roots of the policy that has already been played. The nodes also store the corresponding one-step best response actions A_i^* and value V^* that they induce from agent i given history h_i. Like before, the value of these one-step actions is dependent on the actions taken by the policies at future timesteps. Therefore, nodes have one edge for every action-observation pair $(a_i, o_i) \in \mathcal{A}_i \times \Omega_i$ to other nodes associated with histories $h_i + (a_i, o_i)$. Each of these nodes contains information on how to continue the VSEC in the future.

5.3. DecPOMDPs

In this section we present an algorithm for simultaneously computing the set of best-response policies in a DecPOMDP along with the VSER over these policies. The algorithm is based on point-based, dynamic programming approaches for computing the set of best-response policies in DecPOMDPs (Szer & Charpillet, 2006).

At a high-level, Algorithm 1 is based on a dynamic programming operator (the function **BackupSER**) that computes best-response policies starting at timestep t, along with the intermediate VSECs over them, given the VSECs for policies starting at timestep $t+1$. Using this dynamic programming operator, we can then efficiently compute the full set of best-response policies along with their strategic equivalence classes through backwards induction over a finite-horizon DecPOMDP. Repeated application of this operator corresponds with iteratively computing the nodes of the VSER DAG $\mathcal{G}_i$ from the last timestep to the first.

As hinted by the structure of $\mathcal{G}_i$ in Section 5.2, intermediate VSECs at history h_i^t can be recursively computed in terms of intermediate VSECs from histories that are one-step extensions of h_i^t. The following theorem captures this idea.

Theorem 2 (Bellman backup for strategic equivalence)**.** For all histories h_i^t and policies $\pi_{-i}, \pi'_{-i} \in \Pi_{-i}^{h_i^t}$, we have

Algorithm 1 Computes the valued strategic equivalence relation over best-response policies in the form of $\mathcal{G}_i$.

input DecPOMDP $\mathcal{M}$.
1: **function Main**
2: **for all** $t \in \{T, T-1, \ldots, 0\}$ **do BackupSER**(t)
3: **end function**
4: **function BackupSER**(t)
5: **for** all previous joint policies π^t **do**
6: **for** each agent i and history h_i^t **do**
7: $B(A_i^*, V^*) \leftarrow \emptyset$
8: **for** $[[\pi_{-i}]] \in \mathscr{O}(\mathcal{G}, h_i^t, \pi^t)$ **do**
9: **Bellman**(Q_i^*, π_{-i}, h_i^t)
10: $A_i^* \leftarrow \arg\max_{a_i} Q_i^*(\pi_{-i}, h_i^t, a_i)$
11: $V^* \leftarrow \max_{a_i} Q_i^*(\pi_{-i}, h_i^t, a_i)$
12: $B(A_i^*, V^*)$.add($[[\pi_{-i}]]$)
13: **end for**
14: **for all** (A_i^*, V^*) **do** $\mathcal{G}_i$.addNode($B(A_i^*, V^*)$)
15: **end for**
16: **end for**
17: **end function**
output $\{\mathcal{G}_i\}_{i \in [m]}$

$\pi_{-i} \overset{\text{v}}{\sim}_{h_i^t} \pi'_{-i}$ if and only if,

(i) $\arg\max_{a_i} Q_i^*(\pi_{-i}, h_i^t, a_i) = \arg\max_{a_i} Q_i^*(\pi'_{-i}, h_i^t, a_i)$, and $\max_{a_i} Q_i^*(\pi_{-i}, h_i^t, a_i) = \max_{a_i} Q_i^*(\pi'_{-i}, h_i^t, a_i)$;

(ii) $\pi_{-i} \overset{\text{v}}{\sim}_{h_i^t + (a_i, o_i)} \pi'_{-i}$ for all actions $a_i \in \arg\max_{a_i} Q_i^*(\pi_{-i}, h_i^t, a_i)$ and corresponding possible observations o_i.

Proof. See Appendix B.3 □

As we see below, if we back up the optimal Q-functions, $Q_{\pi_{-i}}^*$ based on future timesteps, we can check condition (i). Since we have already computed VSECs for all future histories, we can check condition (ii).

The optimal Q-functions Q_i^* can be backed up using a recursive computation analogous to the classic Bellman backup operator (see Lemma 3 in Appendix B.3). This operation is defined as the function **Bellman** on line 9 of Algorithm 1. Now we give a detailed explanation of the remaining components of Algorithm 1.

Lines 5 and 6 iterate over each past joint policy π^t, agent i, and possible history h_i^t under π^t for agent i. In deterministic environments, we can reduce line 5 to iterating over policy chains (i.e., sequences of actions), rather than policy trees, since trajectories are deterministic given fixed sequences of actions. All algorithms (including baselines) in our experimental section make use of this optimization. Check Appendix B.5 for more details.